```
Output:
int(1234)
int(1234)
bool(false)
float(123.4)
```

Ah. Straight away, we can already see that things are… quirky. In fact, the entire `filter` module is – how can I put it – a little janky. Validating numbers returns either boolean `false` or… a type-cast value. Which isn't the end of the world; you can base a test on the result of the type: `FILTER_VALIDATE_INT` will always return an integer on success; `FILTER_VALIDATE_FLOAT` will always return a float. Perhaps `FILTER_VALIDATE_BOOL` will be better behaved:

```
var_dump(filter_var(true, FILTER_VALIDATE_BOOL));
var_dump(filter_var('1', FILTER_VALIDATE_BOOL));
var_dump(filter_var('true', FILTER_VALIDATE_BOOL));
var_dump(filter_var('yes', FILTER_VALIDATE_BOOL));
```

```
Output:
bool(true)...
```

All of the above evaluate to boolean `true`, whether the input data is a strictly typed `true` or some string that's roughly equivalent, such as… `"yes"`(!). But not `"ja"`, or `"はい"`, only `"yes"`. Baffling, but there you go. Speaking of baffling…

```
var_dump(filter_var(1.0, FILTER_VALIDATE_BOOL));
var_dump(filter_var('1.0', FILTER_VALIDATE_BOOL));
```

```
Output:
bool(true)
bool(false)
```

The float `1.0` is apparently considered a boolean, but the string `'1.0'` is not. That's barmy. Speaking of barmy, check this out:

```
var_dump(filter_var(false, FILTER_VALIDATE_BOOL));
var_dump(filter_var('false', FILTER_VALIDATE_BOOL));
var_dump(filter_var('0', FILTER_VALIDATE_BOOL));
var_dump(filter_var('no', FILTER_VALIDATE_BOOL));
var_dump(filter_var('unsigned 2003 Toyota Yaris', FILTER_VALIDATE_BOOL));
```